

ADVANCED OPERATING SYSTEMS

Lab 2 — System-wide tracing

Vittorio Zaccaria

Politecnico di Milano

Overview

What you will build

No module this time: three captures of the **same workload**, taken while the kernel keeps running.

```
cat /init > /dev/null
```

- **Part A — syscall events.** Record which system calls `cat` makes and what they return, with tracepoints in a tracefs instance.
- **Part B — function graph.** Record the kernel functions called inside one of those system calls, `do_sendfile`.
- **Part C — perf.** Record the same events into a file with `perf`, then list and aggregate them.

Each part ends with a **checkpoint**: compare your capture with the expected result and answer the questions before moving on. Reset the tracing state between parts: a forgotten setting changes the next capture.

The three parts answer one question: how does BusyBox `cat` really copy a file, and how much of that can each tool show?

Set up the guest

Lab 2 adds no module: `dev-build` is needed only if kernel and initramfs were never built.

```
# Host
T=amd64 E=full make dev-up      # once
T=amd64 E=full make dev-build   # only if never built
T=amd64 E=full make dev-run
```

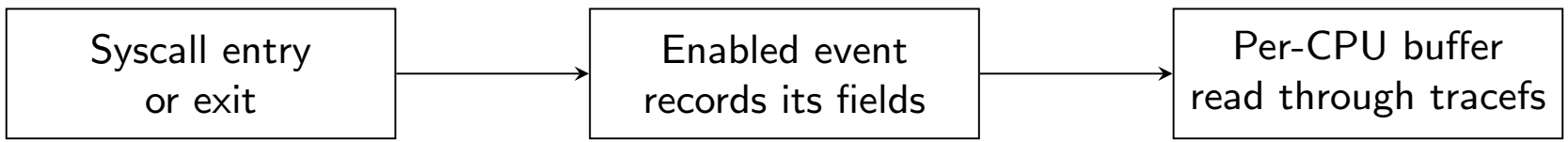
With the browser frontend, run the `AOS: Run QEMU (1 vCPU)` task. Then, in the guest:

```
# Guest
[ -e /dev/null ] || mknod /dev/null c 1 3
[ -d /sys/kernel/tracing/events ] || mount -t tracefs nodev /sys/kernel/tracing
uname -r
wc -c /init
```

Expected: `5.16.0-rc1` and `584 /init`. Keep the **same guest shell** for the whole lab; ignore console lines `... used greatest stack depth ...`. Exit QEMU with `Ctrl-]`, then `x`.

Tracepoints, events and buffers

A **tracepoint** is a predefined instrumentation site. Enabling a syscall tracing event records its fields in a per-CPU tracing buffer.



Tracefs exposes configuration, filters and captured data as files under `/sys/kernel/tracing`. A directory under `instances/` is a separate tracing instance with its own buffer and event settings.

Part A — Syscall events

A1 · Task: trace the file syscalls of `cat`

Record entry and exit of `openat`, `read` and `sendfile64` for one run of the workload, and nothing else.

- Work in a private instance `lab2`. Create it and start from a clean state:

```
# Guest
mkdir -p /sys/kernel/tracing/instances/lab2
T=/sys/kernel/tracing/instances/lab2
export T
echo 0 > "$T/tracing_on"
echo nop > "$T/current_tracer"
echo 0 > "$T/events/enable"
echo > "$T/trace"
```

- Enable the **six** events one by one. Enable both `read` and `sendfile64`: do not assume how this `cat` copies the file.
- Before the run, print the format of `sys_enter_sendfile64` and note its fields.
- Record only the workload process, only while it runs; stop recording before reading the buffer and its statistics for CPU 0.

A1 · Useful tracefs files

Paths are relative to the instance `$T`:

FILE	EFFECT
`events/syscalls/<event>/enable`	`1` enables one event, `0` disables it
`events/enable`	`0` disables every event of the instance
`events/syscalls/<event>/format`	field names and print format of the event
`set_event_pid`	record events only for the listed PIDs; empty write clears
`tracing_on`	`1` starts, `0` stops recording into the buffer
`trace`	read: buffer content; empty write: clear the buffer
`per_cpu/cpu0/stats`	entries, overruns and dropped events on CPU 0

A1 · Record only the workload

The PID of `cat` does not exist before `cat` starts, so it cannot be written in `set_event_pid` in advance. Use this launcher:

```
sh -c '
  echo $$ > "$T/set_event_pid"
  echo 1 > "$T/tracing_on"
  exec cat /init
' > /dev/null
```

- the child shell writes **its own** PID (`$$`) in the filter;
- it starts recording;
- `exec` replaces the shell with `cat` **with the same PID**, so only `cat` is recorded;
- the redirection to `/dev/null` is opened by the parent shell, before recording starts.

Set `tracing_on` back to `0` as soon as the launcher returns.

A1 · Checkpoint

Example from one run (selected fields; PID and timestamps vary):

```
cat-81  [000] ...1.  18.756466: sys_openat(dfd: ffffff9c, filename: 7ffc168c6fa4, ...)
cat-81  [000] ...1.  18.757142: sys_openat -> 0x3
cat-81  [000] ...1.  18.757785: sys_sendfile64(out_fd: 1, in_fd: 3, offset: 0, count: 1000000)
cat-81  [000] ...1.  18.758815: sys_sendfile64 -> 0x248
cat-81  [000] ...1.  18.758835: sys_sendfile64(out_fd: 1, in_fd: 3, offset: 0, count: 1000000)
cat-81  [000] ...1.  18.758847: sys_sendfile64 -> 0x0
```

All records come from one PID; stats show `entries: 6`, `overrun: 0`, `dropped events: 0`. Then reset the instance: `echo 0 > "$T/events/enable"` and `echo > "$T/set_event_pid"`.

Questions.

1. Why is there no `read` event? What do descriptors 3 and 1 refer to?
2. The first `sendfile64` asks for `0x1000000` bytes and returns `0x248`: why is that not an error? Why does `cat` call it a second time, and what does that call transfer?
3. Why does `openat` show a number instead of the path `/init`?

Part B — Function graph

B1 · Task: the call tree of `do_sendfile`

Trace the kernel functions called by `do_sendfile` while the workload runs: only the workload process, at most **3 levels** deep.

- Use the **top-level** instance: on this kernel `instances/lab2` has no `set_graph_function` and no `max_graph_depth`. Leave it clean first:

```
# Guest
T=/sys/kernel/tracing
export T
echo 0 > "$T/tracing_on"
echo 0 > "$T/events/enable"
echo > "$T/set_event_pid"
echo nop > "$T/current_tracer"
echo > "$T/set_ftrace_filter"
```

- Then: choose the root function, limit the depth, select the tracer, clear the buffer, run the workload with the A1 launcher adapted to **function** tracing, stop recording, read the trace and the CPU 0 statistics.

B1 · Useful tracefs files

FILE	EFFECT
`current_tracer`	`function_graph` records call trees; `nop` disables function tracing
`set_graph_function`	root functions of the graph; empty write clears
`max_graph_depth`	maximum nesting depth shown; `0` means no limit
`set_ftrace_filter`	restricts traced functions; empty write clears
`set_ftrace_pid`	PIDs for **function** tracing (`set_event_pid` is for events)
`tracing_on`, `trace`, `per_cpu/cpu0/stats`	as in Part A

Reading the output: one line per call, indentation is nesting. `f() {` opens a call with children, `}` closes it with its duration; `f();` is a leaf. The duration column marks `+` above 10 µs, `!` above 100 µs, `#` above 1 ms.

B1 · Checkpoint

Start of the first root, from one run (durations vary):

```
0)               |  do_sendfile() {
0)               |    __fdget() {
0) + 48.168 us   |      __fget_light();
0)   7.807 us    |      irq_enter_rcu();
0) ! 167.037 us  |      __sysvec_apic_timer_interrupt();
0) + 19.868 us   |      irq_exit_rcu();
0) ! 595.580 us  |    }
```

Expected: **two** `do_sendfile` roots, at most three levels, `overrun: 0`. The timer-interrupt lines may be absent in your run.

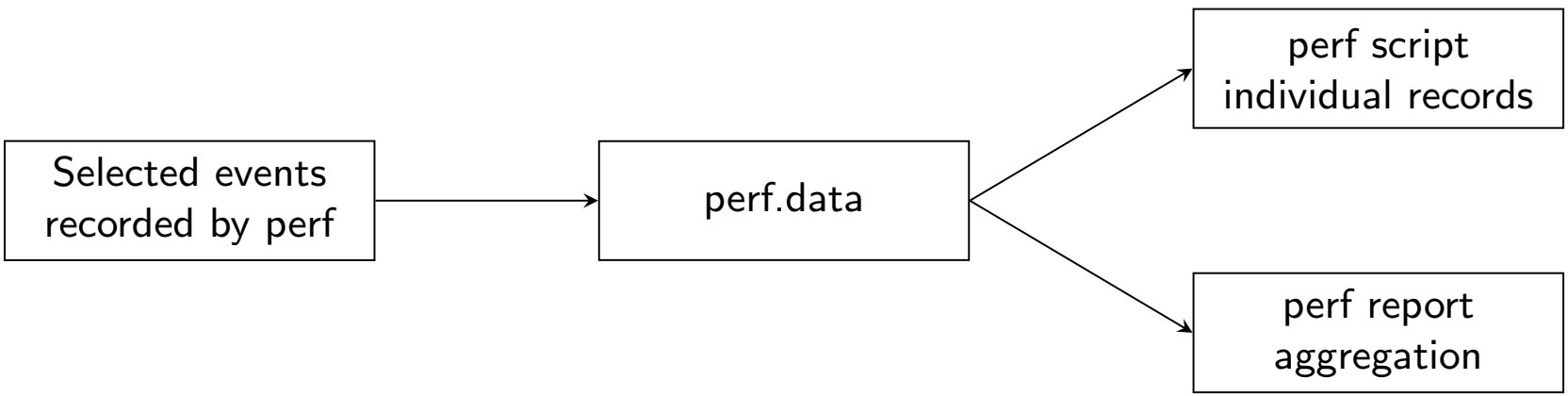
Questions.

1. Why two roots? Which steps does the second run, though it transfers nothing?
2. Does `__fdget` call `__sysvec_apic_timer_interrupt`? Why did the PID filter not remove those lines?
3. Is a parent’s duration the sum of its children’s? Are these real-machine times?

Part C — Recording with perf

Record now, inspect afterwards

Tracefs let you read its buffer directly. **Perf records selected events into a file**, `perf.data`, through the perf event interface and its own buffers; you inspect the file afterwards.



The guest already provides `perf` `5.16.0-rc1`. Syscall events need neither hardware performance counters nor a BPF program.

C1 · Task: record, list, aggregate

First reset Part B (the top-level instance, `T=/sys/kernel/tracing`), then prepare the shell:

```
# Guest
echo nop > "$T/current_tracer"
echo > "$T/set_ftrace_pid"
echo > "$T/set_graph_function"
echo 0 > "$T/max_graph_depth"
mkdir -p /tmp
export PERF_PAGER=cat
perf --version
```

1. Record into `/tmp/lab2-perf.data` entry and exit of the syscalls that **did fire** in Part A, one sample per occurrence, only for the launched workload (not system-wide).

2. Write the individual records to `/tmp/lab2-perf-script.txt` and print it.

3. Write an aggregation by command name, with sample counts and no percentage cut-off, to `/tmp/lab2-perf-report.txt` and print it.

Write to files and `cat` them: this avoids paging and truncation to the terminal width.

C1 · Useful perf options

COMMAND / OPTION	EFFECT
`perf list '<glob>'`	list matching events, e.g. `'syscalls:*sendfile*'`
`perf record ... -- <command>`	run `<command>` and record its events
`-e <event>`	select one event (repeatable); names as `syscalls:sys_enter_openat`
`-c 1`	one sample every occurrence of the event
`-a`	all CPUs and tasks, system-wide: **do not use here**
`-o <file>` / `-i <file>`	output file of `record` / input file of `script`, `report`
`perf script`	print every recorded sample
`perf report --stdio`	text aggregation instead of the interactive browser
`--sort comm`	group samples by command name
`--show-nr-samples`	add a column with the number of samples
`--percent-limit 0`	do not hide entries with a small percentage

C1 · Checkpoint

Expected: `perf record` ends with `Captured and wrote 0.002 MB /tmp/lab2-perf.data (6 samples)`. The script lists, for one `cat` PID, the same sequence as Part A:

```
syscalls:sys_enter_openat
syscalls:sys_exit_openat       0x3
syscalls:sys_enter_sendfile64
syscalls:sys_exit_sendfile64   0x248
syscalls:sys_enter_sendfile64
syscalls:sys_exit_sendfile64   0x0
```

The report has one block per event, with **1, 1, 2, 2** samples, each `100.00%  cat`, and `Total Lost Samples: 0`.

Questions.

1. `100.00%` of what? Is it CPU time?
2. Six samples: how many system calls did `cat` make?
3. Where did the records live in Part A, and where in Part C? What can you do with `perf.data` that you cannot do with the tracefs buffer?

Wrap-up

Done when

- Part A: the `lab2` trace has six records of one PID: `openat` returning 3, then two `sendfile64` returning `0x248` and `0x0`; no overruns, no dropped events.
- Part B: the graph has two `do_sendfile` roots, at most three levels deep.
- Part C: `perf script` shows the same six events; `perf report` shows 1, 1, 2, 2 samples and no lost samples.
- The tracing state is reset:

```
cat /sys/kernel/tracing/current_tracer        # nop
cat /sys/kernel/tracing/set_graph_function    # #### all functions enabled ####
```

- You can answer every checkpoint question: what each tool showed, and what it could not show.

Linux event tracing · Function tracing · Perf examples